HACKUDC 2026 · GRADIANT SECURITY CHALLENGE

G8keeper v2
Criptografía: Estudio completo

Análisis técnico del sistema de cifrado para la defensa del proyecto ante el jurado

Versión vault-v2 **Branch** quantum-security-max **Fecha** Marzo 2026 **Estándar** NIST Cat. 5

ÍNDICE DE CONTENIDOS

1. El problema que resuelve la criptografía
2. Conceptos base desde cero
3. NIST — qué es y por qué importa
4. Niveles de seguridad NIST
5. El sistema completo — 5 fases
6. Resistencia cuántica
7. Comparativa con la industria
8. Principio de Kerckhoffs
9. La cadena completa de un vistazo
10. Frases clave para el jurado

1 El problema que resuelve la criptografía

Un gestor de contraseñas guarda tus contraseñas en disco (en `chrome.storage.local`). Si alguien roba ese fichero, no debe poder leer nada. El objetivo de toda la criptografía es que **ese fichero robado sea inútil sin la contraseña maestra**.

Modelo de amenaza

```
Atacante TIENE:
  ✓  fichero del vault (chrome.storage.local)
  ✓  código fuente completo (open source)
  ✓  potencia de cómputo ilimitada

Atacante NO TIENE:
  ✗  contraseña maestra del usuario
  ✗  las claves criptográficas en memoria
```

CONCLUSIÓN

La única puerta de entrada es la contraseña maestra. Todo el sistema está diseñado para que intentar adivinarla sea computacionalmente imposible incluso con potencia ilimitada.

2 Conceptos base — desde cero

¿Qué es cifrar?

Cifrar transforma datos legibles (plaintext) en datos ilegibles (ciphertext) usando una **clave**. Solo quien tenga la clave puede revertir el proceso.

```
plaintext + clave  →  [algoritmo de cifrado]   →  ciphertext
ciphertext + clave →  [algoritmo de descifrado] →  plaintext
```

¿Qué es una función hash?

Una función hash (SHA-512, por ejemplo) toma cualquier dato y produce una huella digital de tamaño fijo. Tiene tres propiedades cruciales:

- **Determinista:** el mismo input siempre da el mismo output
- **Irreversible:** no se puede reconstruir el input desde el output
- **Efecto avalancha:** cambiar 1 bit del input cambia ~50% del output

```
"password123"  →  SHA-512  →  b3a8e0e1f...  (64 bytes)
"password124"  →  SHA-512  →  f7c2d1a3e...  (64 bytes, completamente diferente)
```

¿Qué es entropía?

La entropía mide la cantidad de incertidumbre de un valor. Se mide en bits.

2^{10} posibilidades = 10 bits | 2^{128} posibilidades = 128 bits | 2^{256} posibilidades = 256 bits

Con 256 bits, aunque pruebes 10^{18} claves por segundo durante la vida del universo, no llegas ni al 0,000000001% del espacio de búsqueda.

3 NIST — qué es y por qué importa

NIST (National Institute of Standards and Technology) es la agencia del gobierno de EE.UU. que publica los estándares criptográficos globales. Cuando NIST aprueba un algoritmo, significa que los mejores criptógrafos del mundo lo han analizado durante años y no han encontrado forma de romperlo.

Estándar NIST	Qué define	Aplica a G8keeper v2
SP 800-132	Uso correcto de PBKDF2 (KDF para contraseñas)	✓
SP 800-38D	AES-GCM (modo de cifrado autenticado)	✓
SP 800-108	HKDF (derivación de múltiples claves)	✓
SP 800-107	Funciones hash seguras (SHA-2)	✓
SP 800-131Ar3	Niveles de seguridad y algoritmos post-2030	✓
FIPS 140-3	Módulos criptográficos seguros	✓ **(Web Crypto)**

TODOS LOS ALGORITMOS DE G8KEEPER V2

están en estos estándares y son los mismos que usa la industria: Google, Apple, 1Password, Bitwarden, el gobierno de EE.UU. para información clasificada.

4 Niveles de seguridad NIST (las "Categorías")

NIST clasifica la seguridad criptográfica por categorías para comparar algoritmos entre sí en escenarios clásicos y post-cuánticos:

Categoría	Equivalente clásico	Equivalente cuántico	Estado
Cat 1	AES-128 (128 bits)	64 bits cuánticos	⚠ **Mínimo aceptable**
Cat 3	AES-192 (192 bits)	96 bits cuánticos	⚠ **Bueno**
Cat 5	**AES-256 (256 bits)**	**128 bits cuánticos**	✓ **Quantum-safe máximo**

G8KEEPER V2 OPERA ÍNTEGRAMENTE EN CATEGORÍA 5

El nivel más alto que NIST define. Es el mismo nivel que se exige para información clasificada del gobierno de EE.UU. y el que Australia (ASD) exige para sistemas críticos después de 2030.

5 El sistema completo — 5 fases

FASE 1

PBKDF2-SHA-512 (KDF)

Transforma la contraseña maestra en material de clave resistente a brute-force

FASE 2

HKDF-SHA-512 (expansión)

Deriva 3 claves independientes con propósito separado

FASE 3

Double AES-256-GCM + AAD

Doble cifrado autenticado con protección contra downgrade

FASE 4

HMAC-SHA-512 verify-first

Verificación de integridad constant-time antes de descifrar

Fase 1 — PBKDF2-SHA-512: de contraseña a material de clave

El problema: Una contraseña como "MiPerro2024!" tiene ~40 bits de entropía. Una clave AES-256 necesita exactamente 256 bits de entropía aleatoria. No puedes usar la contraseña directamente.

La solución: Password-Based Key Derivation Function (PBKDF2), definida en NIST SP 800-132:

```
masterPassword + salt (256 bits, aleatorio)
        |
        ▼
  PBKDF2-SHA-512
  × 1.000.000 iteraciones   ← lento a propósito
        |
        ▼
    512-bit IKM (Input Keying Material)
    (el "embrión" del que nacen todas las claves)
```

Salt — 256 bits (32 bytes), completamente aleatorio:
Generado con `crypto.getRandomValues()`, la fuente de aleatoriedad del sistema operativo, criptográficamente segura. Se guarda junto al vault (no es secreto). Su función: dos usuarios con la misma contraseña producen IKMs completamente distintos. Imposibilita los ataques de **rainbow tables** (tablas precalculadas de hashes).

```
"password123" + salt_usuario_A  →  IKM_A
"password123" + salt_usuario_B  →  IKM_B

IKM_A ≠ IKM_B   ← aunque la contraseña sea idéntica
```

¿Por qué 1.000.000 de iteraciones?
PBKDF2 aplica SHA-512 un millón de veces consecutivas. Cada iteración cuesta tiempo de CPU. El objetivo es que derivar la clave sea deliberadamente lento:

```
Usuario legítimo: 1 vez → ~800 ms → aceptable    |    Atacante: miles de
```

```
millones de veces → inviable
```

Con hardware moderno un atacante puede hacer ~100.000 pruebas/segundo con PBKDF2-SHA-512. Para una contraseña mediocre (2^{40} posibilidades):

```
2⁴⁰ intentos ÷ 100.000 intentos/seg ≈ 125 días
```

Con una contraseña fuerte (2^{60} bits): **350.000 años**.

¿Por qué SHA-512 y no SHA-256?
SHA-512 usa operaciones de 64 bits internamente. Las GPUs y ASICs usados para brute-force están optimizados para operaciones de 32 bits (minería de Bitcoin). SHA-512 es ~2x más caro en ese hardware, multiplicando el coste del atacante. OWASP 2023 recomienda mínimo 600.000 iteraciones de SHA-256. Nosotros usamos **1.000.000 de SHA-512** — significativamente por encima del estándar.

Fase 2 — HKDF-SHA-512: expansión a 3 claves independientes

El problema: Tenemos 512 bits de IKM. Necesitamos 3 claves independientes: una para cifrado interno, una para cifrado externo, una para verificar integridad. Usar la misma clave para múltiples propósitos es una mala práctica criptográfica.

La solución: HKDF (HMAC-based Key Derivation Function), RFC 5869, NIST SP 800-108:

```
512-bit IKM (de PBKDF2)
        |
        ▼
  HKDF-SHA-512
  ┌──────────────────────────────────────────────────┐
  | info = "g8keeper-v2-inner-enc"  →  innerKey (AES-256) |
  | info = "g8keeper-v2-outer-enc"  →  outerKey (AES-256) |
  | info = "g8keeper-v2-hmac-auth"  →  hmacKey  (HMAC-512) |
  └──────────────────────────────────────────────────┘
```

El parámetro `info` actúa como **etiqueta de propósito**. Es matemáticamente imposible que conocer `innerKey` te dé información sobre `outerKey` o `hmacKey` — son independientes por construcción (propiedad de independencia informacional de HKDF, garantizada por RFC 5869 §3).

> **¿POR QUÉ IMPORTA LA SEPARACIÓN DE CLAVES?**
>
> Si se usara una sola clave para cifrar Y para el MAC, ciertos ataques teóricos podrían explotar esa relación. Con claves separadas, comprometer una no compromete ninguna otra. Es el mismo principio que usar cerraduras distintas para la puerta y la caja fuerte.

Fase 3 — Doble AES-256-GCM con AAD

¿Qué es AES-256-GCM?
AES (Advanced Encryption Standard) es el estándar de cifrado simétrico de NIST desde 2001. La variante "256" indica clave de 256 bits. La variante "GCM" (Galois/Counter Mode) añade autenticación al cifrado: AES-GCM es **AEAD** (Authenticated Encryption with Associated Data).

- **Authenticated:** el mensaje viene con un tag de autenticación de 128 bits. Modificar el ciphertext un solo bit hace fallar el tag.
- **Associated Data (AAD):** datos que NO se cifran pero SÍ se autentican. Si un atacante los modifica, el tag falla igualmente.

El flujo de cifrado doble:

```
plaintext (JSON con las contraseñas del usuario)
        |
        |  Layer 1 — AES-256-GCM(innerKey, iv₁)
        |  AAD = { version:2, kdf:"pbkdf2-sha512", iterations:1000000 }
        ▼
  inner_ciphertext  (incluye auth tag de 128 bits)
        |
        |  Layer 2 — AES-256-GCM(outerKey, iv₂)
        ▼
  outer_ciphertext  (incluye auth tag de 128 bits)
        |
        |  HMAC-SHA-512(hmacKey, iv₁ ‖ iv₂ ‖ outer_ciphertext)
        ▼
    hmac_tag  (512 bits)
```

¿Para qué sirve el AAD (Additional Authenticated Data)?
Los metadatos del vault (versión, tipo de KDF, número de iteraciones) se guardan en texto plano — necesitan estar así para poder iniciar el descifrado. Pero si un atacante los modifica, podría forzar el uso de un KDF más débil (ataque de **downgrade**). El AAD soluciona esto: los metadatos se incluyen criptográficamente en el tag de autenticación de AES-GCM. Cambiar `"iterations":1000000` a `"iterations":1` hace fallar el descifrado con error criptográfico.

¿Para qué sirve el doble cifrado?
AES-256-GCM tiene 2^{256} posibles claves. En la práctica es inrompible. El doble cifrado provee **defensa en profundidad**: si en el futuro (extremadamente improbable) se descubriera una debilidad en AES-GCM, la segunda capa independiente con una clave distinta protegería igualmente los datos.

IVs (Initialization Vectors) — 96 bits, aleatorios:
Cada cifrado AES-GCM usa un IV único generado con `crypto.getRandomValues()`. Reutilizar un IV con la misma clave en AES-GCM destruye la confidencialidad y la autenticación. Con IVs aleatorios de 96 bits, la probabilidad de colisión es despreciable (NIST SP 800-38D §8.2.2).

Fase 4 — HMAC-SHA-512: verify-first, constant-time

El HMAC cubre el envelope completo:

```
HMAC-SHA-512(hmacKey,  iv₁ ‖ iv₂ ‖ outer_ciphertext) → tag de 512 bits
```

Cualquier modificación a cualquier parte del vault almacenado — los IVs, el ciphertext, cualquier byte — es detectada inmediatamente.

¿Por qué además del tag de AES-GCM? El HMAC-SHA-512 añade dos garantías adicionales:

- **Verify-first:** el HMAC se verifica *antes* de llamar al descifrado AES-GCM. Si falla, ni tocamos el ciphertext. Esto cierra un ataque teórico llamado **decryption oracle**: si el atacante viera diferencia entre "error de HMAC" vs "error de AES-GCM", podría extraer información por timing. Con verify-first, solo existe una respuesta posible.

- **Constant-time:** se usa `crypto.subtle.verify()`, que realiza la comparación del HMAC en tiempo constante a nivel de implementación nativa. Elimina ataques de **timing side-channel**: un atacante que mide microsegundos de diferencia en el tiempo de respuesta no puede deducir cuántos bytes del tag coinciden.

Fase 5 — Códigos de recuperación

Si el usuario olvida su contraseña maestra, los recovery codes son el último recurso. Hay 4 códigos, cada uno con 256 bits de entropía generados via HKDF-SHA-512, codificados en Base58.

```
Para cada código (×4):
  rcSalt (256-bit, aleatorio)
  PBKDF2-SHA-512(código, rcSalt, 1M iter)  →  rcKey (AES-256)
  AES-GCM(rcKey) cifra el rawIKM de 512 bits  →  encryptedIKM

  SHA-512(codeBytes)  →  hash  (lo que se almacena, nunca el código en plano)
```

Flujo de recovery: usuario introduce código → se verifica contra el hash SHA-512 almacenado → se descifra el rawIKM → con el rawIKM se regeneran las 3 claves via HKDF-SHA-512 → se descifra el vault. El código nunca se guarda en ningún sitio. Solo su huella SHA-512.

6 Resistencia cuántica — explicada correctamente

El algoritmo de Grover

Un ordenador cuántico puede usar el **algoritmo de Grover** para buscar en un espacio de N elementos en √N pasos en lugar de N. Aplicado a criptografía simétrica:

$$2^{256} \text{ claves AES-256} \rightarrow \text{Grover} \rightarrow 2^{128} \text{ operaciones cuánticas efectivas}$$

Esto **no** rompe AES-256 — reduce su seguridad efectiva de 256 bits clásicos a 128 bits cuánticos. 128 bits cuánticos es exactamente la **Categoría 5 de NIST**: el nivel más alto, clasificado como quantum-safe. Por comparación, AES-128 quedaría en 64 bits cuánticos bajo Grover — por eso NIST recomienda migrar a AES-256. G8keeper v2 ya usa AES-256.

El algoritmo de Brassard–Høyer–Tapp para hashes

Reduce la seguridad de SHA-512 de 512 bits clásicos a ~256 bits cuánticos de resistencia a colisiones. 256 bits cuánticos — de nuevo Categoría 5, quantum-safe con margen.

El algoritmo de Shor — NO aplica aquí

El algoritmo cuántico más peligroso es el de **Shor**. Puede romper RSA y ECDH en tiempo polinomial — destruye cualquier criptografía de clave pública basada en factorización de enteros o logaritmo discreto.

> **G8KEEPER V2 NO USA NINGÚN ALGORITMO DE CLAVE PÚBLICA**
>
> No hay RSA, no hay ECDH, no hay curvas elípticas. El sistema es 100% simétrico — estructuralmente inmune a Shor por diseño. Shor solo existe como amenaza si hay un intercambio de claves públicas, y en un vault local no lo hay.

Tabla resumen de resistencia cuántica

Algoritmo	Amenaza cuántica	Seguridad cuántica efectiva	Status
AES-256-GCM	Grover: $2^{256} \rightarrow 2^{128}$	128 bits	✓ **Quantum-safe (Cat 5)**
SHA-512	BHT: ~256 bits	256 bits	✓ **Quantum-safe (Cat 5)**
PBKDF2-SHA-512	Grover sobre hash interno	≥128 bits	✓ **Quantum-safe**
HMAC-SHA-512	Ninguna práctica conocida	256 bits	✓ **Quantum-safe (Cat 5)**
RSA / ECDH	Shor: rotura total	0 bits	✗ **No se usa en G8keeper**

7 Comparativa con la industria

Parámetro	G8keeper v2	Bitwarden	1Password	LastPass (pre-breach)
KDF	PBKDF2-SHA-512	PBKDF2-SHA-256	PBKDF2-SHA-256	PBKDF2-SHA-256
Iteraciones	**1.000.000**	600.000	650.000	100.000
Salt	**256 bits**	128 bits	128 bits	128 bits
Cifrado	Double AES-256-GCM	AES-256-GCM	AES-256-GCM	AES-256-GCM
AAD binding	✓ Sí	✗ No	✗ No	✗ No
Separación de claves (HKDF)	✓ 3 claves	✗ 1 clave	✗ 1 clave	✗ 1 clave
Verify-first MAC	✓ Sí	✗ No	✗ No	✗ No
Quantum-safe	✓ Cat 5	✓ Cat 5	✓ Cat 5	⚠ Cat 3
Dependencias externas	✓ Cero	✗ Varias	✗ Varias	✗ Varias

G8KEEPER V2 SUPERA A BITWARDEN Y 1PASSWORD EN:

Iteraciones del KDF (1M SHA-512 vs 600k SHA-256), tamaño del salt (256 vs 128 bits), separación de claves con HKDF, AAD binding (protección contra downgrade) y verify-first MAC con constant-time. Sin dependencias externas — todo usando la Web Crypto API nativa del navegador.

8 Principio de Kerckhoffs — por qué el open source es una fortaleza

> *Un sistema criptográfico debe ser seguro aunque todo sobre él, excepto la clave, sea de conocimiento público.*
>
> — Auguste Kerckhoffs, 1883

AES, SHA-512 y PBKDF2 son totalmente públicos, documentados en miles de papers, y sometidos al análisis de los mejores criptógrafos del mundo durante décadas. No han podido romperlos. Eso es exactamente lo que los hace confiables.

Seguridad a través de la oscuridad es una ilusión: si un sistema solo es seguro porque nadie sabe cómo funciona, es débil. El momento en que alguien descubre el algoritmo (ingeniería inversa, leak), la seguridad desaparece completamente.

EL OPEN SOURCE COMO VENTAJA COMPETITIVA

Bitwarden es open source. KeePass es open source. Sus auditorías públicas son la razón por la que millones de personas confían en ellos. G8keeper v2 sigue el mismo principio: cualquier experto puede revisar el código y verificar que no hay puertas traseras, bugs ocultos ni malas prácticas criptográficas.

9 La cadena completa — de un vistazo

```
USUARIO introduce su contraseña maestra
            |
            ▼
  ┌──────────────────────────────────────────────────────┐
  |  PBKDF2-SHA-512                                      |
  |  password + salt (256-bit aleatorio)                 |
  |  × 1.000.000 iteraciones                             |
  |  → 512 bits de IKM                                   |
  └──────────────────────────────────────────────────────┘
            |
            ▼
  ┌──────────────────────────────────────────────────────┐
  |  HKDF-SHA-512  (RFC 5869)                            |
  |  IKM + "g8keeper-v2-inner-enc"  →  innerKey (AES-256) |
  |  IKM + "g8keeper-v2-outer-enc"  →  outerKey (AES-256) |
  |  IKM + "g8keeper-v2-hmac-auth"  →  hmacKey  (HMAC-512) |
  └──────────────────────────────────────────────────────┘
            |
            ▼
  ┌──────────────────────────────────────────────────────┐
  |  AES-256-GCM  (innerKey, iv₁, AAD=metadatos)         |
  |  plaintext  →  inner_ciphertext  (protección downgrade) |
  └──────────────────────────────────────────────────────┘
            |
            ▼
  ┌──────────────────────────────────────────────────────┐
  |  AES-256-GCM  (outerKey, iv₂)                        |
  |  inner_ct  →  outer_ciphertext  (defensa en profundidad)|
  └──────────────────────────────────────────────────────┘
            |
            ▼
  ┌──────────────────────────────────────────────────────┐
  |  HMAC-SHA-512  (hmacKey)                             |
  |  iv₁ ‖ iv₂ ‖ outer_ct  →  tag_512bits                |
  |  verify-first + constant-time                        |
  └──────────────────────────────────────────────────────┘
            |
            ▼
  Se guarda en disco:  salt + kdf_params + iv₁ + iv₂
                     + outer_ciphertext + hmac_tag

  La contraseña NO se guarda.  Las claves NO se guardan.
  Solo el salt y los textos cifrados son persistentes.
```

10 Frases clave para el jurado

> *Nuestro stack cumple íntegramente con NIST SP 800-132, SP 800-38D y SP 800-108 — los mismos estándares que usa el gobierno americano para información clasificada.*

> *AES-256 es Categoría 5 de NIST, el único nivel clasificado como quantum-safe bajo el algoritmo de Grover. No usamos criptografía de clave pública, por lo que el algoritmo de Shor — la principal amenaza cuántica — no aplica a nuestra arquitectura por diseño.*

> *Superamos las recomendaciones de OWASP 2023 en cada parámetro: 1M iteraciones de PBKDF2-SHA-512 frente al mínimo recomendado de 600k SHA-256, y salt de 256 bits frente al mínimo de 128 bits.*

> *El open source no es una debilidad — es la razón por la que AES y SHA-512 son confiables. Ambos llevan siendo analizados por la comunidad criptográfica mundial desde 2001 y 2014 respectivamente, con acceso completo al algoritmo. Su fortaleza radica precisamente en esa transparencia.*

> *La única debilidad teórica del sistema es una contraseña maestra débil. Eso no es una debilidad de la criptografía — es una responsabilidad del usuario. La criptografía hace su trabajo correctamente.*

> *Usamos cero dependencias externas. Toda la criptografía se ejecuta sobre la Web Crypto API nativa del navegador — la misma API utilizada por Google, Apple y Mozilla, implementada en código C++ auditado y ejecutada fuera del entorno JavaScript para mayor seguridad.*

G8keeper v2 — Branch `quantum-security-max` — HackUDC 2026 — Gradiant Security Challenge

Todos los algoritmos: PBKDF2-SHA-512 · HKDF-SHA-512 · AES-256-GCM · HMAC-SHA-512 · Web Crypto API nativa